Project 1 Proposal

NYC Street Tree Health Predictor

Intro to Data Science Final Capstone

Prepared for a three-person project team | June 26, 2026

Project type	Small data science app with an embedded classification model
Dataset	NYC Open Data: 2015 Street Tree Census - Tree Data
Target variable	Tree health: Good, Fair, or Poor
Core product	A Streamlit app that predicts tree health and visualizes patterns by species, borough, problems, and location
Positive impact	Help prioritize street-tree care by identifying visible risk patterns and communicating them clearly

Executive summary

This proposal recommends building an NYC Street Tree Health Predictor for the final capstone project. The project is intentionally scoped to be simple enough to finish well while still meeting the course goal: a small data science app with a model embedded that solves a real problem with positive impact. The dataset is public, large, well-documented, and already includes the target outcome needed for classification. The final result can be demonstrated clearly through a notebook, a Streamlit app, and a short presentation.

Recommended decision: proceed with Project 1 because it has the strongest combination of feasibility, clarity, positive impact, and grade potential.

1. Course Fit and Project Rationale

The Brightspace syllabus describes the final capstone as a project that combines the course skills into a small data science app with an embedded model that addresses a real problem with positive impact. This project matches that requirement directly:

- **Python and pandas:** used for importing, cleaning, filtering, and analyzing the tree census dataset.
- **Visualization:** used for health distribution charts, species comparisons, borough patterns, and maps.
- **Classification modeling:** used to predict Good/Fair/Poor tree health from observable features.
- **Streamlit app:** used to turn the model into an interactive tool with user inputs, predictions, charts, and explanation.
- **Positive impact:** supports environmental awareness and helps residents or city teams think about how tree maintenance could be prioritized.

The project is also a strong choice because the target variable already exists in the dataset. That reduces unnecessary complexity and lets the team spend more time on polish, model evaluation, app usability, and explaining limitations.

2. Project Objective

Research question: Can we predict whether an NYC street tree is in Good, Fair, or Poor health using observable information such as species, trunk diameter, borough, stewardship, guards, sidewalk condition, and recorded problems?

End-user question: If a user enters visible information about a street tree, what health category is the model likely to predict, and what factors seem most related to that prediction?

Proposed product: a Streamlit web app where a user selects tree characteristics, receives a predicted health category, sees predicted class probabilities, and explores supporting charts or a map from the dataset.

3. Dataset Plan

Item	Plan
Dataset name	2015 Street Tree Census - Tree Data
Source	NYC Open Data: https://data.cityofnewyork.us/d/uvpi-gqnh
Approximate size	About 683,000 tree records in the full dataset; the team can use a manageable sample for faster app performance.
Target variable	health: Good, Fair, Poor
Candidate features	tree_dbh, spc_common, steward, guards, sidewalk, problems, boroname, latitude, longitude
Unit of analysis	One street tree record
Why it works well	The dataset is public, concrete, easy to explain, and includes both model features and map coordinates.

The team should keep the first version narrow: predict health from a clean set of common fields, then add polish after the baseline model and app are working. If the full dataset is slow, the notebook can analyze the full data while the app uses a saved sample or trained model artifact.

4. Proposed Methodology

Stage	What the team will do	Expected output
Data import	Load the NYC Open Data file or the prepared sample; confirm columns, row count, missing values, and target labels.	Clean starting dataset and data dictionary notes.
Cleaning	Drop rows without health; standardize missing categorical values; keep only useful features; create simple problem indicators if needed.	Model-ready dataframe.
Exploratory analysis	Show health distribution, top species, health by borough, trunk diameter patterns, common problems, and a map sample.	4-6 clear visuals with short interpretations.
Baseline model	Train a simple Logistic Regression or Decision Tree pipeline with one-hot encoding for categorical variables.	Baseline accuracy and macro-F1.
Improved model	Train a Random Forest classifier and compare performance to the baseline.	Improved model metrics and feature importance.
Evaluation	Use a train/test split; report accuracy, macro-F1, confusion matrix, and class-level performance.	Honest assessment of model strengths and weaknesses.
App deployment	Build a Streamlit interface with inputs, prediction output, probability chart, map/EDA section, and limitations.	Interactive final capstone app.

5. App Design

The app should be polished but not overcomplicated. The goal is to make the data science workflow understandable to a grader in a few minutes.

- **Home section:** project description, dataset source, and why tree health matters.
- **Prediction section:** inputs for species, borough, diameter, stewardship, guards, sidewalk condition, and observed problems; output predicted health and probabilities.
- **Insights section:** charts for health distribution, borough comparison, top species, and problems versus health.
- **Map section:** sample of tree records colored by health or predicted risk.
- **Model explanation section:** model type, features used, performance metrics, feature importance, and limitations.

A simple but clean user flow is better than a complicated app with bugs. The app should load quickly, avoid confusing inputs, and explain that the model supports triage and learning rather than making official maintenance decisions.

6. Evaluation and Success Criteria

Area	A-level target
Technical correctness	Data cleaning and modeling pipeline runs end-to-end without manual fixes.
Model evaluation	At least two models compared; metrics include accuracy and macro-F1 because the classes are imbalanced.
Visualization quality	Charts are labeled, readable, and connected to specific insights.
App quality	Streamlit app is easy to use, has clear inputs, shows prediction probabilities, and includes context.

Area	A-level target
Impact explanation	Proposal and presentation clearly connect the tool to tree-care prioritization and urban environmental health.
Limitations	Team acknowledges class imbalance, snapshot data, correlation not causation, and possible measurement bias.

7. Final Deliverables

- **Notebook:** data import, cleaning, exploratory data analysis, feature engineering, model training, evaluation, and limitations.
- **Streamlit app:** interactive prediction tool plus supporting visuals and model explanation.
- **GitHub repository:** README, app file, notebook, requirements.txt, trained model or training instructions, and sample data if needed.
- **Presentation:** 6-8 slides covering problem, data, EDA, model, app demo, impact, limitations, and next steps.
- **Optional model card:** one-page summary of what the model does, how it should and should not be used, metrics, and limitations.

8. Three-Person Work Delegation

The work should be divided by ownership area, but all members should review the final notebook, app, and presentation so the project feels consistent. The suggested delegation below assumes a three-person team with rotating peer review.

Team member	Primary ownership	Specific tasks	Final outputs
Member 1: Data and EDA Lead	Dataset preparation and exploratory analysis	Download/load data; define columns; clean missing values; create model-ready dataframe; produce health distribution, species, borough, problem, and map visuals; write dataset/EDA notes.	Clean dataset or sample; EDA notebook section; 4-6 visuals; data limitations notes.
Member 2: Modeling and Evaluation Lead	Model pipeline, training, comparison, and metrics	Build preprocessing pipeline; train baseline model; train Random Forest; evaluate accuracy, macro-F1, confusion matrix, and feature importance; save model artifact if useful.	Modeling notebook section; comparison table; selected final model; evaluation explanation.
Member 3: App, README, and Presentation Lead	Streamlit interface, repository polish, and final storytelling	Build app layout; connect model to user inputs; add probability chart and map/insights section; write README; create slide deck; coordinate final demo script.	Streamlit app; README; requirements.txt; final slides; demo script.

Collaboration rules

- **Daily check-in:** each person posts what they finished, what is blocked, and what they will do next.
- **Shared naming conventions:** agree early on column names, feature list, and file names to prevent merge issues.
- **Peer review:** each member reviews at least one section owned by another member before submission.
- **Demo rehearsal:** run the full app and presentation together at least once before the final deadline.

9. Suggested Timeline

Phase	Deadline target	Team focus
Phase 1: Setup	Day 1	Create GitHub repo, confirm dataset access, agree on feature list, and make a simple project board.
Phase 2: Data and EDA	Days 2-3	Clean data, create visuals, write initial insights, and identify missing-value or imbalance issues.
Phase 3: Modeling	Days 4-5	Train baseline and improved model, evaluate results, and choose final model for the app.
Phase 4: App build	Days 5-6	Build Streamlit app, connect model, add chart/map sections, and test inputs.
Phase 5: Polish	Days 7-8	Finalize README, notebook explanations, slides, limitations, and demo script.
Phase 6: Rehearsal	Final day before submission	Run everything from scratch, fix errors, rehearse presentation, and verify links/files.

10. Risks and Mitigation

Risk	Why it matters	Mitigation
Class imbalance	Most trees may be labeled Good, so accuracy alone can look better than the model really is.	Report macro-F1 and confusion matrix; explain class imbalance honestly.
Slow app performance	The full dataset is large and may make Streamlit slow.	Use a sample for app visuals and save a trained model artifact.
Too many categories	Species and problems can create many one-hot encoded columns.	Limit to top species plus Other, or use only common categories in the first version.
Overclaiming impact	The model is based on observational data and cannot prove what causes tree health.	Frame the app as a triage and learning tool, not an official decision system.
Last-minute integration issues	Notebook, model, and app may not connect cleanly if built separately.	Integrate early with a simple version, then improve features after it works.

11. Recommended Final Scope

The best final scope is a reliable, polished version of the project rather than an overly ambitious one. The team should prioritize:

- A clean notebook that tells the full data science story from data to model.
- A model comparison between one baseline and one improved classifier.
- A Streamlit app that makes one prediction task easy and visually clear.
- A thoughtful explanation of positive impact, limitations, and next steps.
- A practiced demo where every team member can explain their contribution.

If time remains, the strongest optional enhancements are a feature-importance chart, a model-card page, and a map showing a sample of trees colored by health. These extras improve clarity without changing the core project.

Sources

- NYU Brightspace syllabus for Principles of Data Science I: final capstone goal and course tools.
- NYC Open Data, 2015 Street Tree Census - Tree Data: https://data.cityofnewyork.us/d/uvpi-gqnh
- Streamlit documentation: https://streamlit.io/

Bottom line: Project 1 is the safest high-grade choice because it is feasible, directly aligned with the capstone requirement, easy to demonstrate, and meaningful to a real urban-environment problem.